

Gated / Paid Telegram Bot

Feature 4 of 13 — free preview, then sell full access via Telegram Stars or a card payment.

No payment processor required

Telegram Stars built-in

Optional Stripe card upsell

1 What this does

A different bot process from the daily poster (feature 1) — this one stays running and listens for messages. When someone sends `/start`, they see 3 free sample tools with a "🔓 Unlock All Tools" button. Tapping it opens a native Telegram payment sheet charging **Telegram Stars** — Telegram's own in-app currency for digital goods. No Stripe account, no payment processor sign-up, nothing to configure beyond picking a price. Once paid, the buyer is remembered and instantly gets a button into the full Mini App (feature 3).

Why Stars, not a normal payment gateway?

Telegram requires digital goods/services sold inside a bot to use Stars, not external processors, per their bot platform policy. Stars are bought by users with real money (via Apple/Google in-app purchase) and you withdraw them as real currency from your bot's balance in @BotFather. It also means zero setup — no merchant account needed to start selling today.

2 Where this bot has to run

This one can't run on Vercel or GitHub Actions

Unlike the daily poster, this bot must stay running continuously to listen for messages — Vercel's serverless functions shut down between requests, and GitHub Actions jobs have a time limit. Run it on: your own always-on computer, a small VPS (~\$5/mo), or a free-tier host like Railway or Render that supports long-running processes.

3 Set the price and start it

1. Add `TELEGRAM_BOT_TOKEN` to your environment (same bot/token as feature 1).
2. Optionally set `TELEGRAM_STARS_PRICE` (defaults to 99 Stars, roughly \$2 — adjust to taste).
3. Run: `node scripts/telegram-bot.mjs`
4. Message your bot `/start` to test the free preview and unlock flow.

4 Optional: add a card payment option

To let buyers pay with a real card instead of Stars (useful outside Telegram's ecosystem, e.g. for real-currency accounting):

STEP 1

Create a Payment Link

In your Stripe Dashboard
→ Payment Links →
create one for your
unlock price.

STEP 2

Add the webhook

Stripe Dashboard → Developers → Webhooks →
Add endpoint:
`https://yourdomain.com/api/webhooks/stripe` ,
event: `checkout.session.completed` .

STEP 3

Set 3 env vars

`STRIPE_PAYMENT_LINK` ,
`STRIPE_SECRET_KEY` ,
`STRIPE_WEBHOOK_SECRET`
— deploy, and a "Pay with
card" button appears
automatically.

5 Persisting who's unlocked

Set `UPSTASH_REDIS_REST_URL` / `UPSTASH_REDIS_REST_TOKEN` (free tier at console.upstash.com) so unlock status survives
bot restarts. Without it, unlocks are remembered in a local file only — fine for testing, risky for production.

6 Where the code lives

File	Purpose
<code>scripts/telegram-bot.mjs</code>	The long-running bot: preview, Stars checkout, unlock delivery
<code>app/api/webhooks/stripe/route.ts</code>	Handles Stripe's payment confirmation (optional path)
<code>scripts/lib/kv.mjs</code>	Tracks which Telegram users are unlocked

7 Troubleshooting

Problem	Fix
Bot doesn't respond to /start	Confirm the script is actually still running — check your host's process logs
Unlock button does nothing	Check the terminal running the bot for a Telegram API error printed to console
Payment succeeds but no confirmation message	Check your host allows outbound network requests, and that the bot process didn't crash mid-payment
Unlocks disappear after restart	Set up Upstash (step 5) — the local file fallback doesn't survive most host redeploys